

MAINCRAFTS TECHNOLOGY

Internship Project Report

Introduction to Embedded Systems & IoT Device Design

Task 1: Smart Temperature Monitoring and Alert System

Field	Details
Intern Name	Priyadarshini S
Intern ID	MT8929
College	Alva's Institute of Engineering and Technology
Domain	Embedded Systems & Internet of Things (IoT)
Organisation	Maincrafts Technology
Internship Period	01 October 2026 to 15 October 2026 (2 Weeks)
Simulation Platform	Tinkercad Circuits

1. Introduction to Embedded Systems

1.1 What is an Embedded System?

An embedded system is a computer-based system designed to perform a specific function or a set of functions within a larger electronic or mechanical system. Unlike general-purpose computers, embedded systems are developed for dedicated applications. They typically consist of a controller, sensors, memory, and input/output interfaces.

In this project, the Arduino UNO acts as the controller, the TMP36 sensor measures temperature, and an LED and a buzzer provide the visual and audible alerts.

1.2 Applications of Embedded Systems

Embedded systems are widely used across many fields, including:

- Smart home systems
- Automotive systems
- Healthcare devices
- Robotics
- Agricultural automation
- Industrial automation
- Consumer electronics

1.3 Microcontroller vs Microprocessor

Feature	Microcontroller	Microprocessor
Function	Designed mainly for dedicated control applications	Used mainly for general-purpose computing
Memory	Usually integrated on the chip	Generally requires external memory
Peripherals	Usually integrated on the chip	Often externally connected
Power	Generally lower	Generally higher
Applications	Embedded systems and automation	Computers and high-performance systems

2. Component Study

2.1 Microcontrollers

Arduino UNO: A microcontroller-based development board used for embedded-system prototyping and automation. In this project, it reads the TMP36 sensor, processes the temperature value, compares it with the threshold, and controls the LED and buzzer.

ESP32: A microcontroller platform commonly used for IoT applications and wireless sensor systems. It is suitable for IoT monitoring, smart-home applications, wireless communication, and automation.

Raspberry Pi Pico: A compact microcontroller board suitable for embedded-system projects. Its applications include sensor-based systems, robotics, automation, and embedded prototypes.

2.2 Basic Sensors

Temperature Sensor (e.g., TMP36): Measures the temperature of its surroundings and produces a proportional analog voltage. This project uses the TMP36 to obtain the temperature reading.

Motion Sensor (e.g., PIR sensor): Detects movement in its surroundings. Applications include security systems, automatic lighting, and motion detection.

Light Sensor (e.g., LDR): Detects the intensity of light. Applications include automatic lighting systems and light-level monitoring.

3. Mini Project: Smart Temperature Monitor

3.1 Objective

The objective of this project is to design and simulate a Smart Temperature Monitoring and Alert System using an Arduino UNO and a TMP36 temperature sensor. The system continuously monitors the temperature and activates an LED and a buzzer when the temperature reaches or exceeds a predefined threshold of 30°C.

This corresponds to the Smart Temperature Monitor option specified in the Task 1 brief.

3.2 Components Used

Component	Purpose
Arduino UNO	Main controller
TMP36	Temperature sensing
LED	Visual alert
Piezo buzzer	Audible alert
220 Ω resistor	LED current limiting
Breadboard	Circuit assembly
Jumper wires	Electrical connections

3.3 Circuit Connections

Component	Connection
TMP36 left pin	5V
TMP36 middle pin	A0
TMP36 right pin	GND
LED (anode)	D8 through a 220 Ω resistor
LED (cathode)	GND
Piezo buzzer (+)	D9
Piezo buzzer (-)	GND

3.4 Circuit Diagram

The circuit was built and simulated in Tinkercad Circuits, as shown in Figure 1.

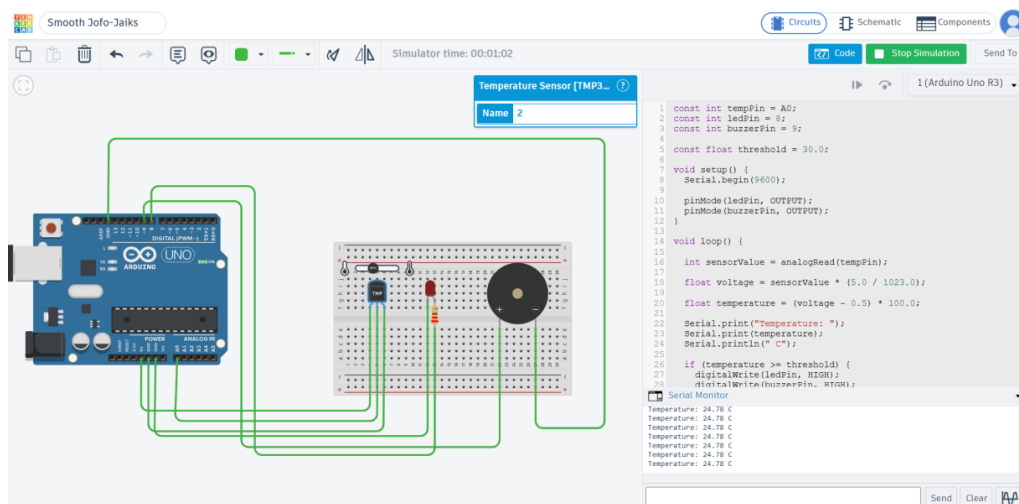


Figure 1: Smart Temperature Monitoring System simulated using Tinkercad Circuits

Tinkercad project link: <https://www.tinkercad.com/things/bhBBumLcG4L-smart-temperature-monitor-simulation>

4. Working Principle

The system operates according to the following sequence:

Temperature → TMP36 → Arduino UNO → Threshold Comparison → Alert

1. The TMP36 senses the surrounding temperature and produces an analog voltage corresponding to it.
2. The Arduino UNO reads this signal through analog pin A0.
3. The analog reading is converted into temperature in degrees Celsius.
4. The Arduino compares the measured temperature with the 30°C threshold.
5. If the temperature is below 30°C, the LED and buzzer remain OFF.
6. If the temperature is 30°C or above, the LED and buzzer are turned ON.
7. The measured temperature is displayed continuously on the Serial Monitor.

5. Arduino Source Code

```
// Smart Temperature Monitoring and Alert System
// Platform : Arduino UNO R3 with TMP36 temperature sensor

const int tempPin   = A0;    // TMP36 output connected to analog pin A0
const int ledPin    = 8;     // Alert LED connected to digital pin D8
const int buzzerPin = 9;     // Piezo buzzer connected to digital pin D9

const float threshold = 30.0; // Alert threshold in degrees Celsius

void setup() {
    Serial.begin(9600);        // Start serial communication at 9600 baud

    pinMode(ledPin, OUTPUT);   // Configure LED pin as output
    pinMode(buzzerPin, OUTPUT); // Configure buzzer pin as output
}

void loop() {

    // Read the raw analog value (0 to 1023) from the TMP36
    int sensorValue = analogRead(tempPin);

    // Convert the raw reading into voltage (0 to 5 V)
    float voltage = sensorValue * (5.0 / 1023.0);

    // Convert voltage to temperature in Celsius
    // TMP36: 500 mV offset at 0 C and 10 mV per degree C
    float temperature = (voltage - 0.5) * 100.0;

    // Display the temperature on the Serial Monitor
    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.println(" C");

    // Compare the measured temperature with the threshold
    if (temperature >= threshold) {
        digitalWrite(ledPin, HIGH);    // Turn LED ON
        digitalWrite(buzzerPin, HIGH); // Turn buzzer ON
    }
    else {
        digitalWrite(ledPin, LOW);      // Turn LED OFF
        digitalWrite(buzzerPin, LOW);   // Turn buzzer OFF
    }

    delay(1000); // Wait 1 second before the next reading
}
```

6. Code Explanation

6.1 Pin Declaration

The constants tempPin, ledPin, and buzzerPin define the Arduino pins connected to the temperature sensor (A0), the LED (D8), and the buzzer (D9) respectively.

6.2 Threshold

The constant threshold is set to 30.0 and represents the temperature, in degrees Celsius, at which the alert is triggered.

6.3 Setup Function

The setup() function starts serial communication at 9600 baud so that readings can be viewed on the Serial Monitor, and configures the LED and buzzer pins as outputs.

6.4 Sensor Reading and Temperature Conversion

The function analogRead() returns a value between 0 and 1023 corresponding to the voltage at A0. This value is converted into voltage and then into temperature using the following relations:

$$\text{Voltage (V)} = \text{Sensor Value} \times (5.0 / 1023.0)$$

$$\text{Temperature (}^{\circ}\text{C)} = (\text{Voltage} - 0.5) \times 100$$

The TMP36 outputs 0.5 V at 0°C and changes by 10 mV for every 1°C, which is why the offset of 0.5 V is subtracted and the result is multiplied by 100.

6.5 Alert Condition

The statement if (temperature >= threshold) checks whether the temperature has reached or exceeded 30°C. If the condition is true, both the LED and the buzzer are switched ON using digitalWrite(); otherwise they are switched OFF. A delay of one second between readings keeps the output readable.

7. Simulation Results

The project was simulated using Tinkercad Circuits. The sensor temperature was varied in the simulator and the behaviour of the outputs was observed under two conditions.

7.1 Test Case 1: Normal Temperature

Condition: Temperature below 30°C.

Observation: The Serial Monitor displayed a steady reading of 24.78°C. The LED and buzzer remained OFF.

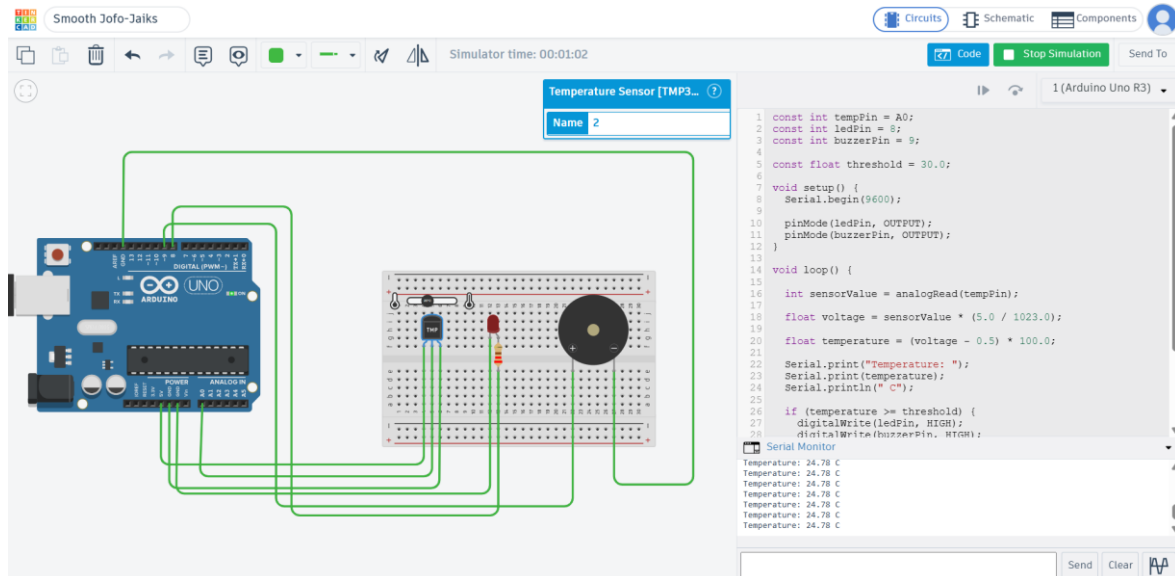


Figure 2: Normal temperature condition (24.78°C): LED and buzzer OFF

7.2 Test Case 2: High Temperature

Condition: Temperature above 30°C.

Observation: The temperature was raised to approximately 47°C and the Serial Monitor displayed 46.77°C. The LED turned ON and the buzzer was activated.

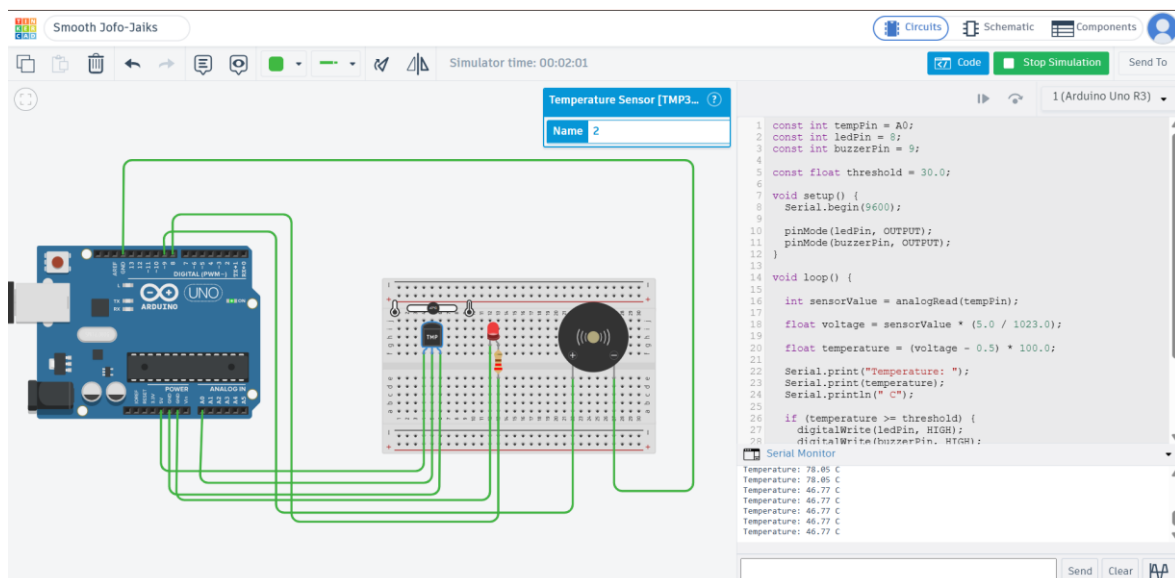


Figure 3: High-temperature alert condition (46.77°C): LED and buzzer ON

7.3 Summary of Results

Condition	Temperature	LED	Buzzer
Below 30°C	24.78°C	OFF	OFF
Above 30°C	46.77°C	ON	ON

The simulation successfully demonstrated that the system activates the visual and audible alerts when the temperature crosses the predefined threshold, and keeps them OFF otherwise.

8. Real-World Applications

The system can be adapted for the following applications:

- Smart home temperature monitoring
- Industrial equipment temperature monitoring
- Agricultural and environmental monitoring
- Controlled healthcare environments
- Detection of overheating in electronic equipment

9. Future Enhancements

The prototype can be extended by replacing the Arduino UNO with an ESP32 and adding Internet connectivity. Possible improvements include:

- Cloud-based temperature monitoring
- Mobile notifications
- Real-time temperature graphs
- Data logging
- Remote monitoring
- Adjustable temperature thresholds

A future version could follow the architecture below:

Temperature Sensor → ESP32 → Wi-Fi → Cloud → Mobile Dashboard

10. Conclusion

The Smart Temperature Monitoring and Alert System was successfully designed and simulated using an Arduino UNO and a TMP36 temperature sensor in Tinkercad Circuits. The system continuously monitors temperature and compares the measured value with a predefined threshold of 30°C. When the temperature reaches or exceeds the threshold, the LED and buzzer are activated to provide an immediate warning.

The project provided practical exposure to embedded systems, sensors, analog signal processing, microcontroller programming, and basic IoT-oriented system design.